Cadence: Machine-Learning Methods & Interpretability

How AI powers the ankle freeze-of-gait garment

Companion notes to the Accuracy Worksheet | ENGF0031 Scenario 2 (Smart Clothing)

Purpose. This companion explains *how* machine learning and AI are used in the Cadence detector, defines the accuracy metrics we report and why, and shows how we interpret what the model has learned. Cadence senses gait from a single ankle accelerometer and fires a vibrotactile cue when it detects a *freeze of gait* (FoG). Every figure and number here is computed from the public **Daphnet** FoG benchmark using **leave-one-subject-out** evaluation — a model is never tested on a patient it trained on — so the numbers estimate performance on a *new* wearer. Nothing is fabricated; the scripts that generate each figure are listed on the last page.

1 How machine learning & AI are applied in this project

1.1 The clinical problem, and why we use ML

Freeze of gait is a sudden, involuntary inability to step — the feeling that the feet are "glued to the floor." It is a leading cause of falls in Parkinson's disease. A well-timed rhythmic *cue* (here, a buzz on the ankle) can help the wearer break the freeze and step off. The engineering task is therefore: *detect a freeze within a second or two, from one on-body accelerometer, in real time.*

The classic hand-built rule is the *Freeze Index* — the ratio of power in a "freeze" band (3–8 Hz) to a "locomotion" band (0.5–3 Hz). It works on average, but the threshold that suits one patient's gait misfires on another's. This is exactly where **supervised machine learning** helps: we show the model many labelled windows of real accelerometer data and let it *learn* the freeze pattern — including patient-to-patient variation — rather than hard-coding one rule. That learned pattern is the AI behind Cadence.

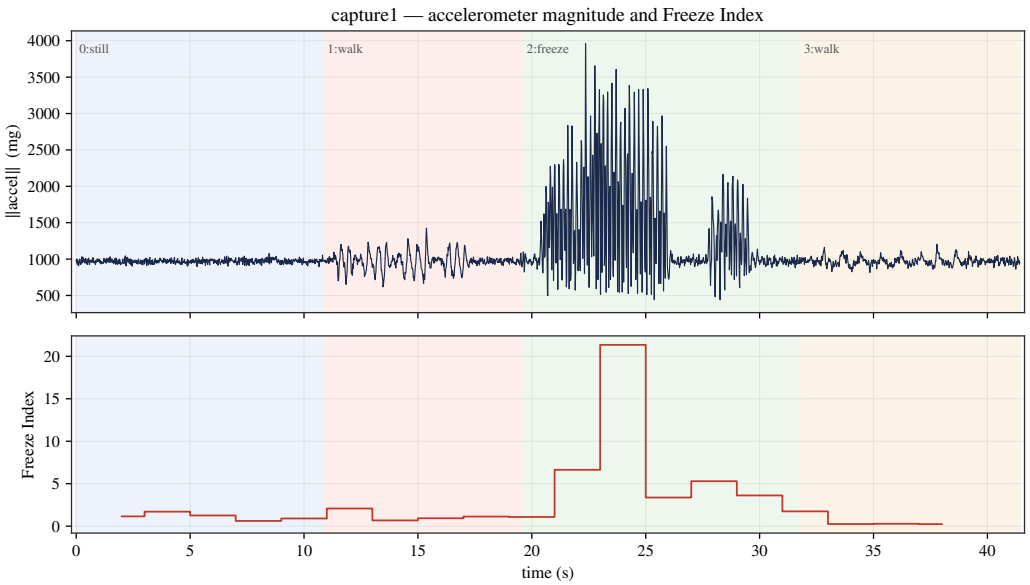

Figure 1. The raw signal the detector sees, from our own ∼41 s ankle capture (phases shaded: still, walk, freeze, walk). *Top:* accelerometer magnitude; *bottom:* the textbook Freeze Index. During the freeze the signal becomes a high-frequency tremor-in-place and the Freeze Index spikes — but note the index is noisy, which is why a learned model does better.

1.2 Two models, two different jobs

We deliberately use *two* models, because "accurate" and "explainable" pull in different directions:

- **FoGNet — a 1-D convolutional neural network (∼18k parameters)** reads the raw 3×256 accelerometer window (4 s at 64 Hz) and learns its own filters end-to-end. *This is the deployed detector*, and its LOSO sensitivity/specificity are the headline result.
- **A 9-feature random forest (RF)** is trained on interpretable, hand-engineered features (band powers, jerk, magnitude statistics, dominant frequency, Freeze Index). It is *not* deployed — it is our "*glass box*": because each input has a physical meaning, we can use SHAP, permutation importance and partial dependence (§4) to see *what in the signal* drives a "freeze" decision, and to sanity-check the CNN.

1.3 The whole picture at a glance

Figure 2 maps the five things ML/AI touches in this project: sensing, the two models, the decision logic, honest evaluation, and interpretability. Figure 3 then traces the live closed loop a single window travels through.

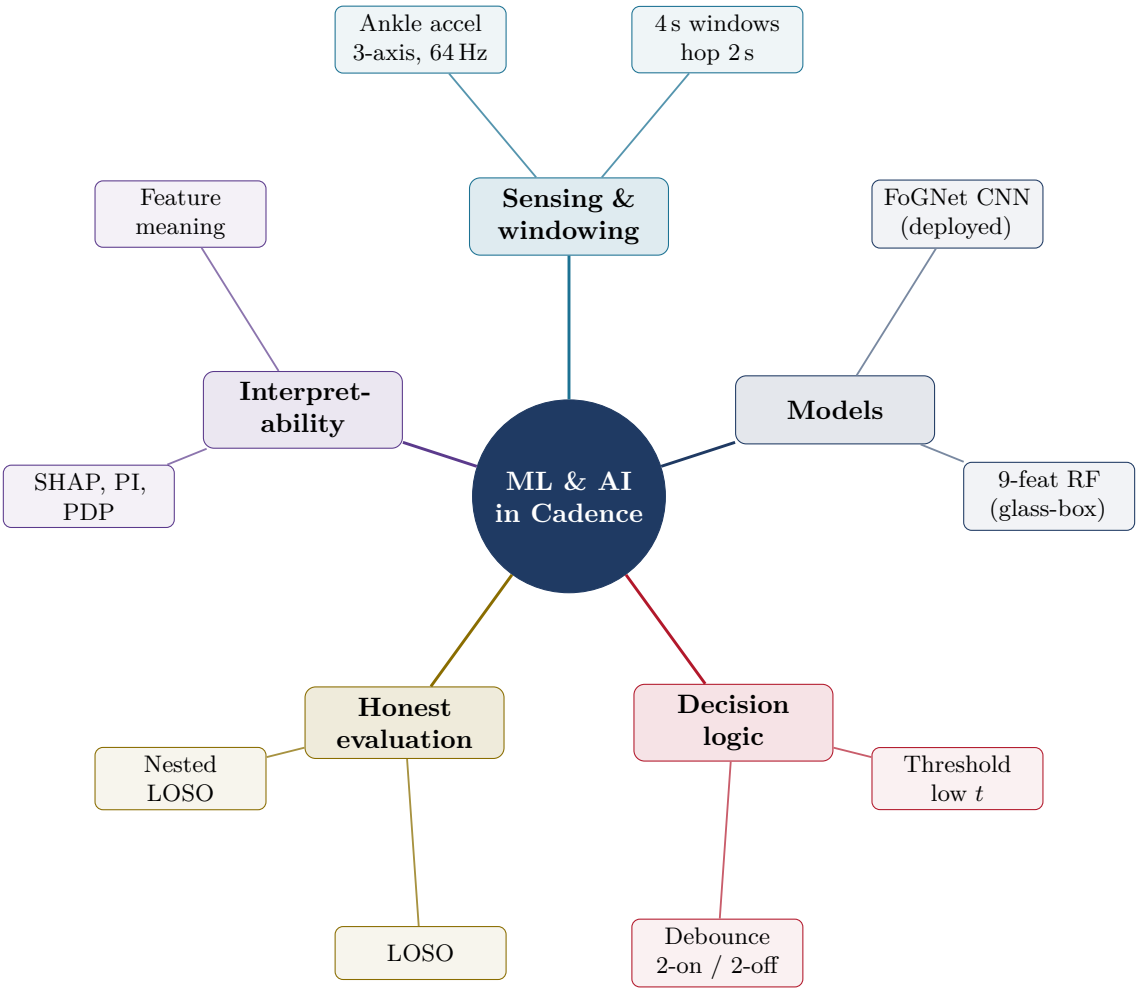

Figure 2. Mind map of where machine learning and AI enter the Cadence project. Each coloured branch is one theme; the outer nodes are the concrete pieces. Sensing feeds the two *models*; the model's probability is turned into an action by the *decision logic*; *honest evaluation* keeps the reported numbers fair; *interpretability* explains what was learned.

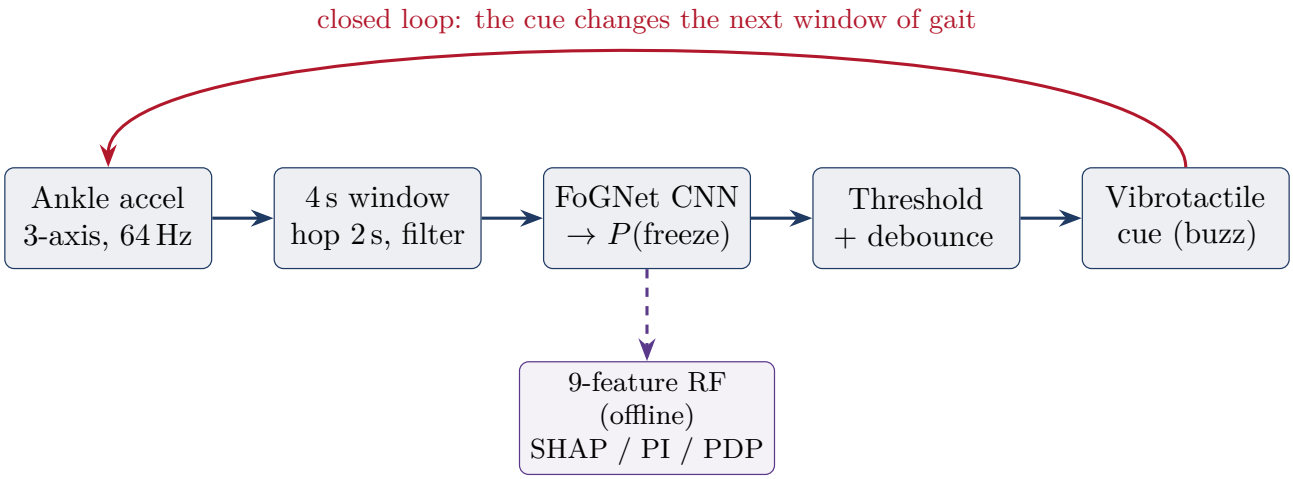

Figure 3. The live pipeline one window travels through. Raw ankle acceleration is buffered into a 4 s window, the CNN outputs a freeze probability, a *low* threshold plus debouncing (assert after 2 positive windows, release after 2 clear — matching the firmware) decides whether to buzz. The buzz feeds back into the wearer's gait, closing the loop. The random forest sits *off* the live path: it exists only to explain the signal.

2 Categorising windows & finding the best parameters

2.1 Categorising = drawing a boundary on a probability surface

A classifier does not output "freeze" or "not" directly — it outputs a *probability* $P(\text{freeze})$ for each window. We "categorise" by comparing that probability to a *threshold* t: buzz if $P(\text{freeze}) \geq t$. Figure 4 shows this surface over two example features. The coloured shading is the probability; a contour line of constant probability *is* the decision boundary, and choosing t simply slides that line. Because freezes are rare (only 9.5% of windows), the useful boundary is the *low* green contour ($t \approx 0.1$), not the textbook 0.5: at 0.5 we would only flag the small, very-high-jerk pocket and miss most freezes.

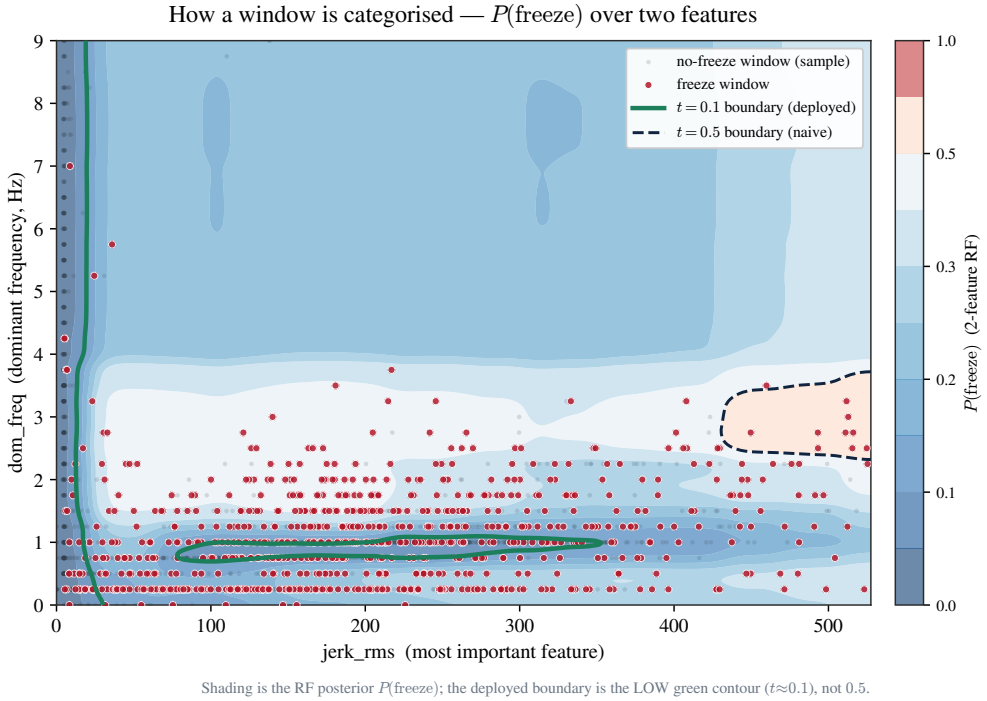

Figure 4. How a window is categorised. The random forest's posterior $P(\text{freeze})$ over two real features (`jerk_rms` and `dom_freq`), with every real window overlaid (freezes in red, a sample of non-freezes as a haze). Freezes concentrate at high jerk and low dominant frequency. Lowering the threshold from the dashed 0.5 line to the solid green $t \approx 0.1$ line enlarges the "freeze" region — trading specificity for the sensitivity we need.

2.2 Finding the best parameters — there are two kinds

"Best parameters" means two different things, and we tune them separately.

(a) Model complexity (hyper-parameters). How deep should each tree be? How many samples per leaf? Too simple and the model underfits; too complex and it memorises the training subjects and fails on a new one. We grid-search these with *subject-grouped* 5-fold cross-validation (Figure 5) and keep the held-out optimum (here `max_depth` = 8, `min_samples_leaf` = 16). Grouping the folds by subject is the honest part: the validation patients are never in the training folds.

(b) The decision threshold. Given a trained model, where do we put t? Sweeping it (Figure 6) trades sensitivity against specificity. We pick the point that maximises *Youden's* J (= sensitivity + specificity − 1); for the interpretable RF this is $t^* \approx 0.10$ (sensitivity 0.75, specificity 0.76). The threshold-free ranking quality is summarised by ROC-AUC = 0.82 and PR-AUC = 0.33.

2.3 Why honest selection matters

Both the threshold *and* the hyper-parameters must be chosen on data that is **disjoint by subject** from the data we score them on. If we scanned every threshold on the whole pooled set and then quoted

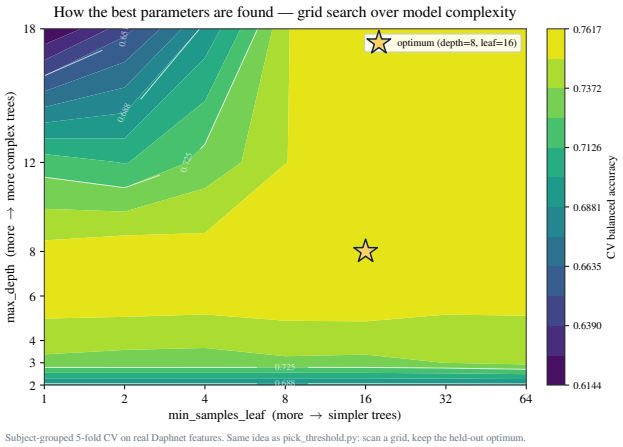

Figure 5. Hyper-parameter search. Subject-grouped CV balanced accuracy over tree depth × leaf size; the star is the held-out optimum. Both over- and under-complex corners score worse.

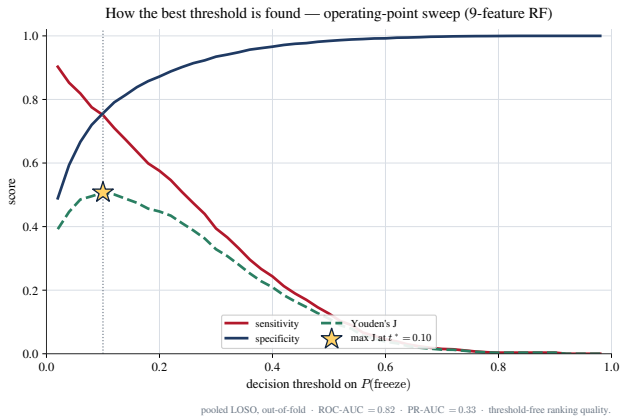

Figure 6. Threshold search. As t rises, sensitivity falls and specificity rises; Youden's J peaks at $t^* \approx 0.10$. This is exactly what `pick_threshold.py` automates per subject.

the best number from that *same* set, we would be tuning on the test data and over-reporting. Our `pick_threshold.py` does *nested* LOSO — it chooses t on the other patients, then scores the held-out one — so the reported sensitivity and specificity are what a genuinely *new* patient should expect.

3 The 15 most useful accuracy metrics

3.1 Why "accuracy" is the wrong headline

Freeze windows are rare (9.5%). A lazy model that always says "no freeze" is therefore ∼90% *accurate* while catching *zero* freezes — useless for a cueing garment (Figure 7). Accuracy hides exactly the failure we care about, so we headline **sensitivity** (freezes caught) and **specificity** (calm gait correctly left alone) instead.

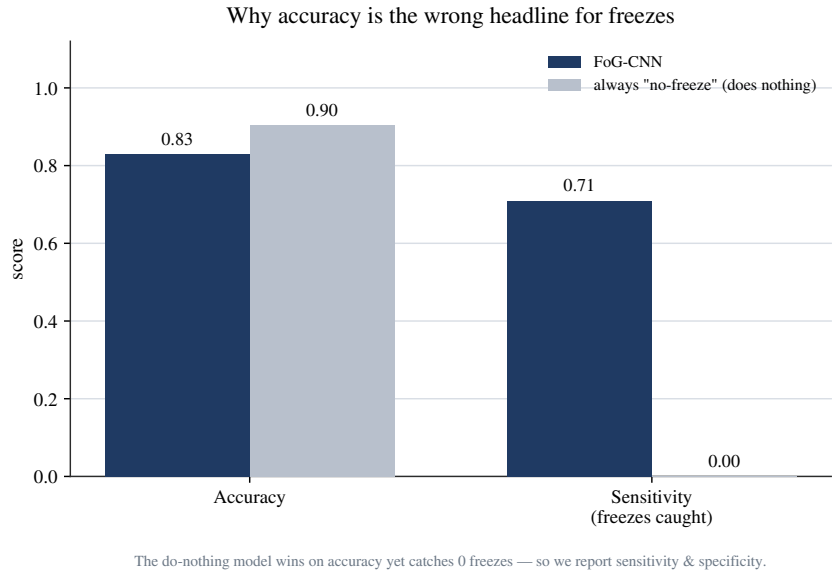

Figure 7. The accuracy trap. A do-nothing model "wins" on accuracy (0.90 vs 0.83) yet has 0.00 sensitivity — it never cues a single freeze. This is why our headline metric is sensitivity & specificity, not accuracy.

3.2 The confusion matrix — where every metric comes from

All of the metrics below are arithmetic on four counts, with *freeze* as the positive class. From the deployed CNN's pooled LOSO predictions: TP = 607 (freezes caught), FP = 1285 (false buzzes), FN = 248 (missed freezes), TN = 6842 (calm correctly ignored), over $N = 8982$ windows.

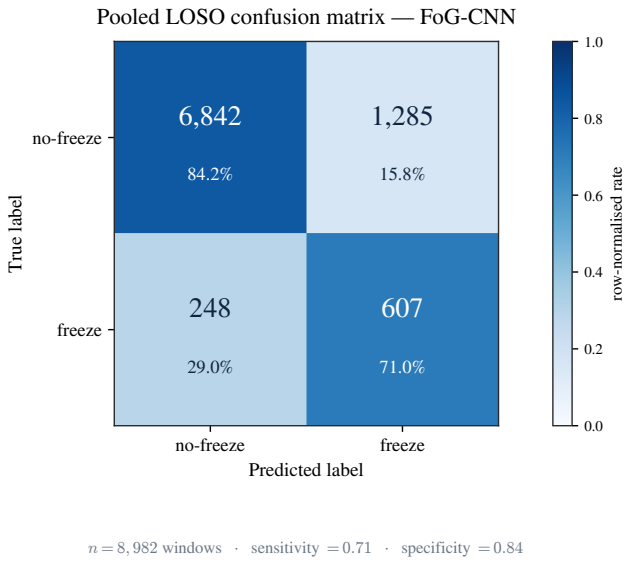

Figure 8. Confusion matrix for the deployed FoG-CNN (pooled LOSO). Rows are the truth, columns the prediction; the off-diagonal cells are the two error types every metric trades against.

3.3 The fifteen metrics

Notation: TPR = sensitivity, TNR = specificity, PPV = precision; positive class = freeze. For MCC, let N_P = TP+FN (actual freezes), N_N = TN+FP, $N_{\hat{P}}$ = TP+FP (predicted freezes), $N_{\hat{N}}$ = TN+FN. For κ, p_o is observed agreement (= accuracy) and p_e is agreement expected by chance.

Metric	Formula	Value	What it tells you (and why it matters here)
1. Sensitivity (recall, TPR)	$\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$	**0.710**	Fraction of real freezes the garment catches. **Headline #1** — a missed freeze means no cue when the patient needs it.
2. Specificity (TNR)	$\frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}$	**0.842**	Fraction of calm/normal gait correctly left alone. **Headline #2** — low specificity means nuisance buzzes that annoy and get the device switched off.
3. Precision (PPV)	$\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$	0.321	Of all the times we buzz, the fraction that were truly freezes. Looks low, but that is the rare-class arithmetic (see prevalence): worth less here than recall.
4. NPV	$\frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FN}}$	0.965	When the device stays silent, the chance the wearer really was not freezing — reassuringly high.
5. F1 score	$\frac{2\,\mathrm{PPV}\cdot\mathrm{TPR}}{\mathrm{PPV}+\mathrm{TPR}}$	0.442	Harmonic mean of precision and recall — one number when you must balance the two, but it ignores the (large) true-negative count.
6. F2 score	$\frac{5\,\mathrm{PPV}\cdot\mathrm{TPR}}{4\,\mathrm{PPV}+\mathrm{TPR}}$	0.571	Like F1 but weights *recall* four times more than precision — the right emphasis for a safety cue where misses cost more than false alarms.
7. Balanced accuracy	$\frac{1}{2}(\mathrm{TPR}+\mathrm{TNR})$	0.776	Plain accuracy re-weighted as if the two classes were equal in size — the honest "accuracy" for imbalanced data.
8. MCC	$\frac{\mathrm{TP}\cdot\mathrm{TN}-\mathrm{FP}\cdot\mathrm{FN}}{\sqrt{N_P\,N_N\,N_{\hat{P}}\,N_{\hat{N}}}}$	0.397	Correlation between prediction and truth using all four cells; +1 perfect, 0 chance, −1 inverted. The most robust single score under class imbalance.
9. Cohen's κ	$\frac{p_o-p_e}{1-p_e}$	0.358	Agreement *above chance*. Guards against being impressed by accuracy that a coin-flip with the same base rate would also reach.
10. Youden's J	$\mathrm{TPR}+\mathrm{TNR}-1$	0.552	Sensitivity + specificity −1. A single "how good is this operating point" score; maximising it is how we pick the threshold (§2).
11. LR+	$\frac{\mathrm{TPR}}{1-\mathrm{TNR}}$	4.49	A buzz multiplies the odds that the wearer is truly freezing by ∼4.5× — the diagnostic "weight" of a positive.
12. LR−	$\frac{1-\mathrm{TPR}}{\mathrm{TNR}}$	0.345	Silence multiplies the odds of freezing by ∼0.34 — i.e. makes a freeze about 3× less likely.
13. Miss rate (FNR)	$\frac{\mathrm{FN}}{\mathrm{TP}+\mathrm{FN}}$	0.290	Fraction of freezes we miss (= 1− sensitivity). The error we most want to drive down for a fall-prevention device.
14. Fall-out (FPR)	$\frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}}$	0.158	Fraction of calm gait wrongly buzzed (= 1− specificity) — the comfort/battery cost of the device.
15. Prevalence	$\frac{\mathrm{TP}+\mathrm{FN}}{N}$	0.095	Base rate of freeze windows. Explains why precision is modest and why accuracy is the wrong headline.

Bottom line. The deployed FoG-CNN reaches **sensitivity** 0.71 **/ specificity** 0.84 on unseen patients (LOSO). We accept modest precision because, for a safety cue, catching freezes (recall) matters more than the occasional extra buzz.

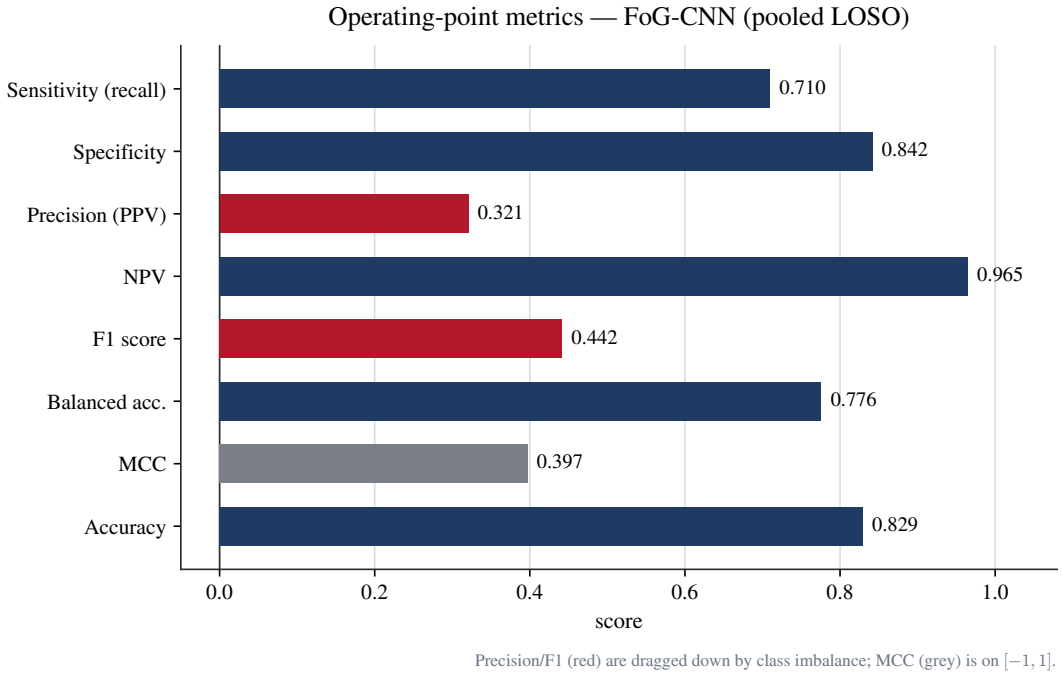

Figure 9. Eight of the key metrics at the deployed operating point. Sensitivity and specificity (navy, headline) sit well above chance; precision and F1 (red) are pulled down by the 9.5% prevalence, not by a weak detector; MCC (grey) lives on $[-1, 1]$.

4 Interpreting the model: SHAP, permutation importance, partial dependence

These three tools answer *different* questions about the glass-box random forest, so they need not agree — and where they differ is itself informative.

- **Permutation importance (PI)** — "how much does held-out performance *drop* if I randomly shuffle this one feature?" A global, model-level, performance-based ranking.
- **SHAP** — "for *this* window, how much did each feature push the prediction up or down?" A local, additive attribution; averaging its magnitude gives a global view.
- **Partial dependence (PDP)** — "as I sweep one feature across its range, how does the average predicted P(freeze) move?" Shows the *shape* and *direction* of an effect.

4.1 Global importance: what the model leans on

Both rankings (Figures 10 and 11) put `jerk_rms` — the rate of change of acceleration — at or near the top, and SHAP makes it dominant (about twice the next feature). Physically this fits: a freeze is a trembling, shuffling "attempt to move" at the ankle, which is rich in jerk. Note a subtlety: `total_power` and `freeze_power` rank *high* in permutation importance but are *flat* in PDP below — they matter through *interactions* with other features, not through a simple one-feature trend.

4.2 Direction & spread: which way each feature pushes

The SHAP beeswarm (Figure 12) colours each window by its feature value. For `jerk_rms`, high values (red) sit on the positive side — they push the prediction *towards* freeze — and its spread is the widest, confirming it is the strongest, most consistent driver.

4.3 Shape of the effect, and the Freeze-Index punchline

Partial dependence (Figure 13) shows `jerk_rms` rising *monotonically* — more jerk, higher freeze probability — while `total_power` and `freeze_power` are nearly flat on their own (consistent with their effect

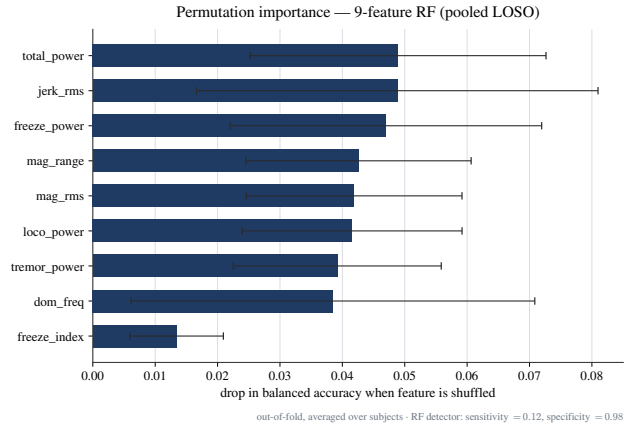

Figure 10. Permutation importance: drop in balanced accuracy when each feature is shuffled. Top features overlap within their error bars; the Freeze Index ranks *last*.

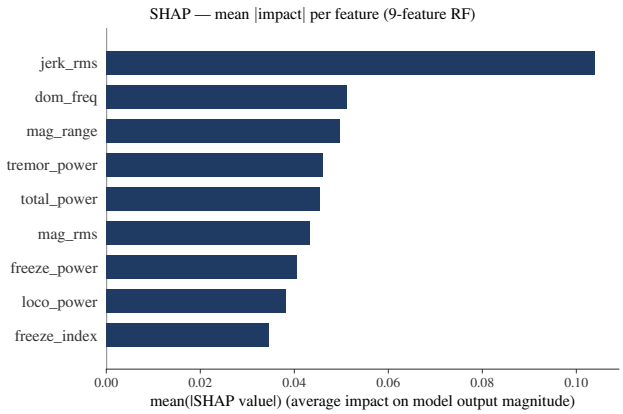

Figure 11. Mean |SHAP| per feature. `jerk_rms` dominates (∼2× the next); the Freeze Index is again last.

being interaction-driven).

Finally, the teaching punchline: the hand-built **Freeze Index ranks last** in *both* PI and SHAP. This is *not* because the 3–8 Hz band is irrelevant — it is because its ingredients (`freeze_power` and `loco_power`) are *already separate features*. A model that can combine them itself finds the fixed textbook *ratio* redundant. This is a clean, honest example of feature redundancy — and of why a learned detector can outgrow a hand-crafted index, which is the whole reason we use ML here.

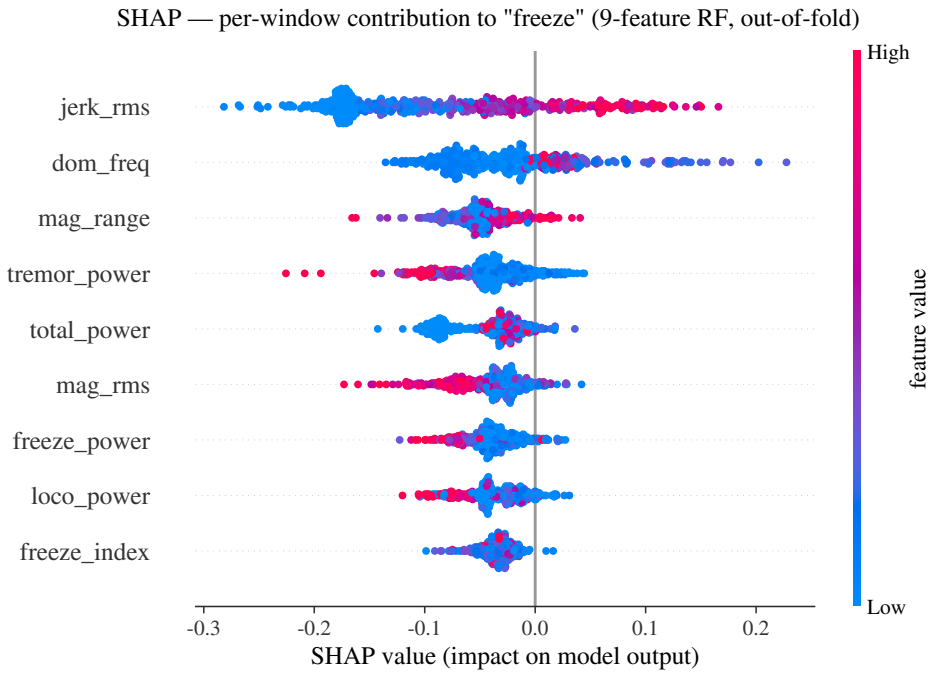

Figure 12. SHAP beeswarm: each dot is one window; horizontal position is that feature's contribution to the freeze score, colour is the feature value (red high, blue low). High `jerk_rms` (red, far right) pushes towards "freeze"; its band is by far the widest.

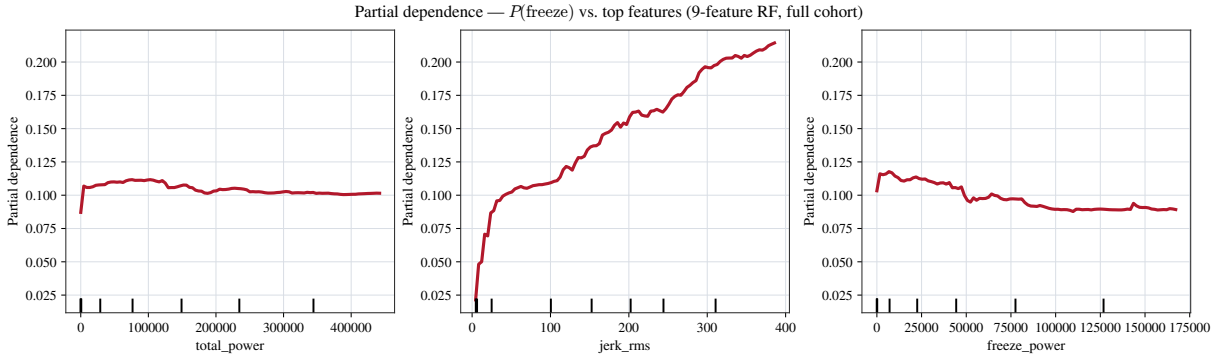

Figure 13. Partial dependence of P(freeze) on the top three features. `jerk_rms` (centre) climbs monotonically from ∼0.02 to ∼0.21; `total_power` and `freeze_power` are flat marginally — their importance acts through interactions, which a single-feature plot cannot show.

5 How the CNN learns its filters

The figures so far used the interpretable random forest. This section opens the *deployed* model — the 1-D convolutional network FoGNet — and shows the four ideas that make it learn: its architecture, the loss it minimises (binary cross-entropy), the algorithm that minimises it (gradient descent), and what a real training run actually looks like.

5.1 Architecture: from a raw window to a freeze probability

FoGNet takes the raw 3×256 accelerometer window and passes it through three convolutional blocks — each *learning* its own filters — then pools and feeds two small dense layers, ending in two scores that a softmax turns into $P(\text{freeze})$ (Figure 14). Convolution is the key idea: a short filter slides along the signal and responds when it meets a local pattern (a tremor burst, a heel-strike), so the network *discovers* what a freeze looks like instead of being told. Each max-pool halves the time axis so deeper filters see longer spans; global average pooling then collapses each of the 64 channels to a single number, which makes the decision robust to *where* in the window the freeze falls. The whole network is just 18,050 trainable weights — small enough to run on the garment's microcontroller.

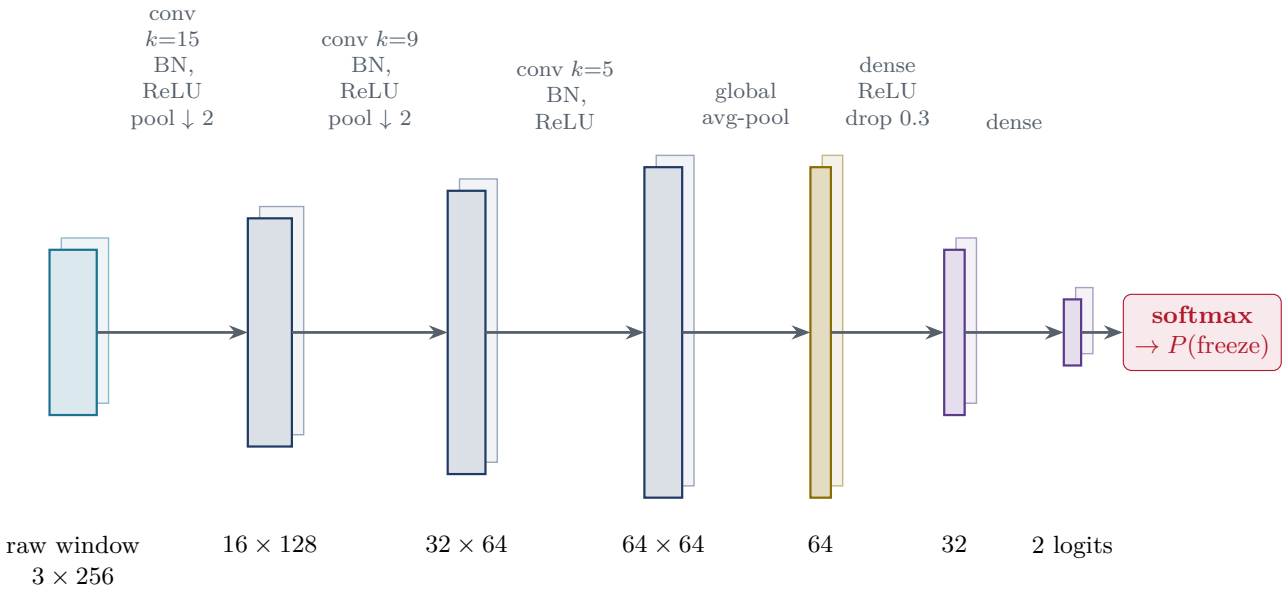

Figure 14. FoGNet, the deployed detector, end to end. Block height grows with the number of channels (the representation the network builds up); the tensor shape (channels × time) is printed beneath each stage. The three convolutional blocks (navy) are the *feature extractor* — each learns its own filters, and the two max-pools halve the time axis ($256 \to 128 \to 64$). Global average pooling (amber) reduces each of the 64 channels to one number; two dense layers (purple) form the *classifier head*, ending in two logits that the softmax turns into $P(\text{freeze})$. Total: 18,050 learnable weights.

5.2 The loss: binary cross-entropy (log-loss)

Training needs a single number that says *how wrong* the model currently is, so that "less wrong" has a direction to move in. That number is the **loss**. For two classes the natural choice is **binary cross-entropy** (log-loss): for a window with true label $y \in \{0, 1\}$ and predicted freeze probability $\hat{p}$,

$$\mathcal{L} = -\big[\, y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \,\big].$$

If the truth is "freeze" ($y = 1$) this is $-\log \hat{p}$ — near zero when the model confidently says freeze, and *exploding* when it confidently says calm (Figure 15, left). Being confidently *wrong* is punished far more than being unsure, which is exactly the incentive a safety detector should have.

Where the probability comes from (the honest detail). FoGNet's last layer outputs *two* raw scores (logits), one per class; a **softmax** turns them into probabilities that sum to one. With only two

classes the softmax equals the **sigmoid** of the score difference $z = z_{\text{freeze}} - z_{\text{calm}}$ (Figure 15, right), and minimising two-class cross-entropy on the softmax is mathematically identical to minimising binary cross-entropy on that single freeze probability. We implement the two-logit form (PyTorch `CrossEntropyLoss`) for one practical reason: it lets us *weight the rare freeze class* directly in the loss — without that weighting the model would happily ignore the 9.5% of windows that matter most. The sigmoid also shows why training can stall: once a score is very large or very small the curve is flat, its gradient is ≈ 0, and that window stops teaching the model anything.

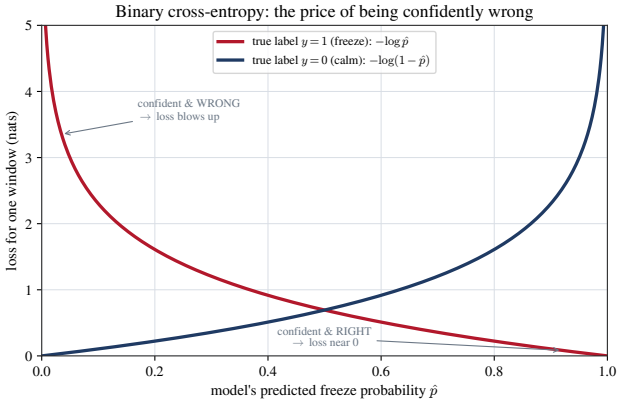

Figure 15. Binary cross-entropy for one window. The loss is small only when the model is confident *and* correct; confidently wrong sends it sharply upward.

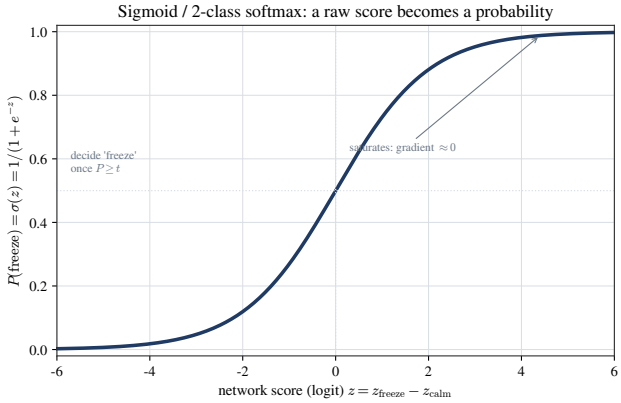

Figure 16. The sigmoid (= 2-class softmax) maps any real score to a probability in $(0, 1)$. Flat tails mean a vanishing gradient; the threshold t is just a vertical cut on this axis.

5.3 Learning by gradient descent

The loss depends on every one of the 18,050 weights at once. **Training** means nudging all of them, a little at a time, in the direction that reduces the loss fastest — the *negative gradient*, computed for every weight simultaneously by **back-propagation**. The size of each nudge is the **learning rate**: too small and training crawls; too large and it overshoots the minimum and zig-zags or diverges (Figure 17). Cadence uses **AdamW**, an adaptive optimiser that gives each weight its own effective step size, with a cosine schedule that shrinks the rate as training proceeds. Real loss surfaces are not the tidy bowl drawn here — they live in 18,050 dimensions and are bumpy — but the picture (go downhill; mind your step size) is exactly right.

5.4 Watching it learn — and knowing when to stop

Figure 18 is a *real* FoGNet run on the Daphnet ankle data. The training loss (red) falls steadily — the model fits the training windows better every epoch. But the metric we actually care about, sensitivity on a *held-out* patient (navy), tells a different story: it climbs, peaks early (epoch 6 here), then drifts *down* as the network begins to memorise its training subjects rather than learn the general freeze pattern. This is **over-fitting**, and the cure is **early stopping**: we keep the checkpoint at the validation peak (gold), not the final epoch. (Our trainer scores checkpoints by sensitivity $+ 0.3 \times$specificity, so it leans toward catching freezes.) A single held-out patient is noisy — the headline **sensitivity** 0.71 **/ specificity** 0.84 is the average over *all* patients under leave-one-subject-out, which is the honest estimate for a new wearer.

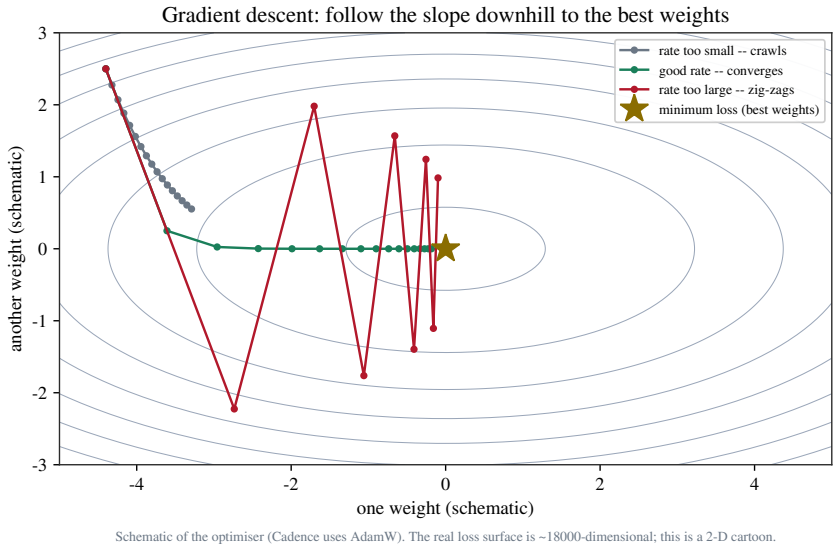

Figure 17. Gradient descent on a 2-D toy loss bowl. From the same start, too small a rate (grey) barely moves, a good rate (green) slides smoothly to the minimum, and too large a rate (red) overshoots and zig-zags. The real optimiser is AdamW in ∼18 000 dimensions; this is a schematic of the idea.

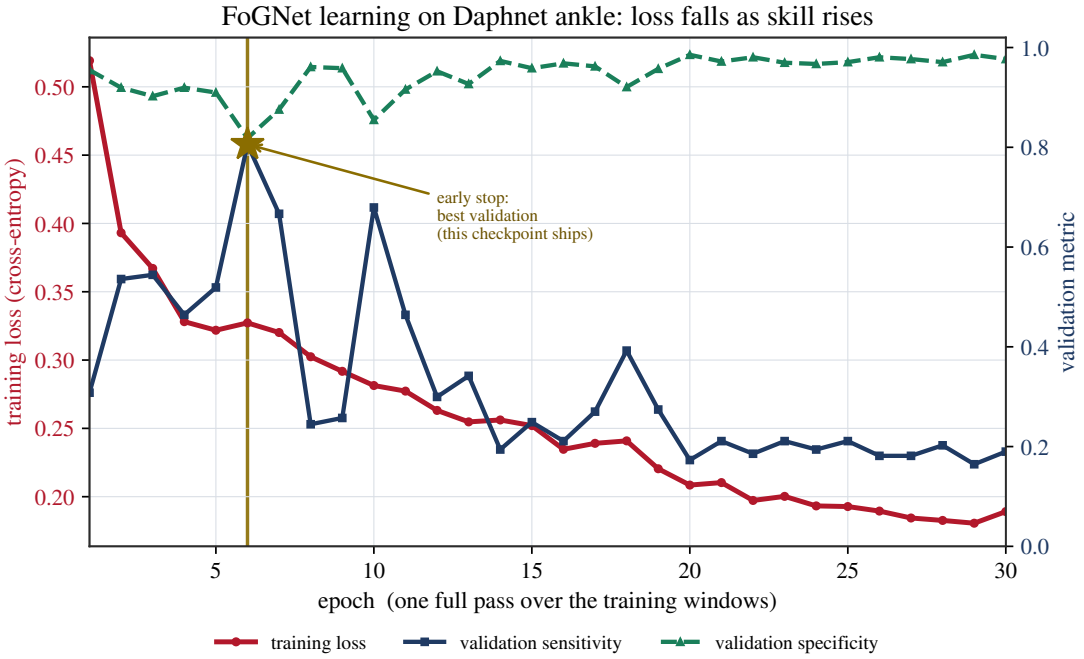

Figure 18. A real FoGNet training run (validation = held-out subject S05). The training loss (red, left axis) keeps falling while held-out sensitivity (navy) and specificity (green, right axis) plateau and then degrade — the textbook over-fitting signature. The gold marker is the early-stopped checkpoint that is actually deployed, not epoch 30. One subject is noisier than the pooled-LOSO headline of 0.71/0.84.

6 More diagnostic graphs for the detector

Section 2 introduced the decision threshold; this section shows the full set of diagnostics behind it. All seven plots use the interpretable 9-feature random forest evaluated by pooled **leave-one-subject-out** — every window is scored by a model that never saw its own patient (the *out-of-fold* probabilities). As in §4 we diagnose the glass box, because its inputs are physically meaningful; the plots characterise *ranking quality*, *probability honesty*, *per-patient generalisation*, and the *feature structure* the model has to untangle.

6.1 Ranking quality, independent of any threshold

Before fixing a threshold we can ask: does the model *rank* freeze windows above calm ones? The ROC curve (Figure 19) sweeps every threshold and plots sensitivity against the false-alarm rate; the area under it (AUC = 0.82) is the probability that a random freeze window scores higher than a random calm one. Because freezes are rare, the precision–recall curve (Figure 20) is the more honest view: its baseline is the 9.5% prevalence, and the average precision (AP = 0.33) sits well above that floor. The gold star marks the deployed operating point ($t \approx 0.10$).

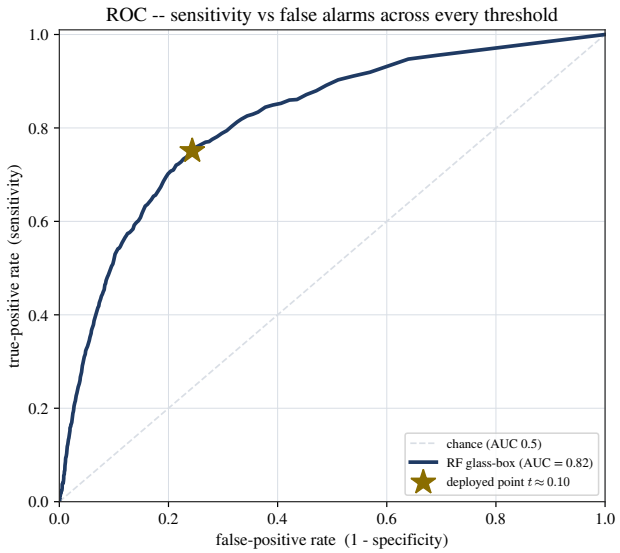

Figure 19. ROC: sensitivity vs false alarms across all thresholds. AUC = 0.82 — the chance a random freeze outranks a random calm window. The diagonal is coin-flip.

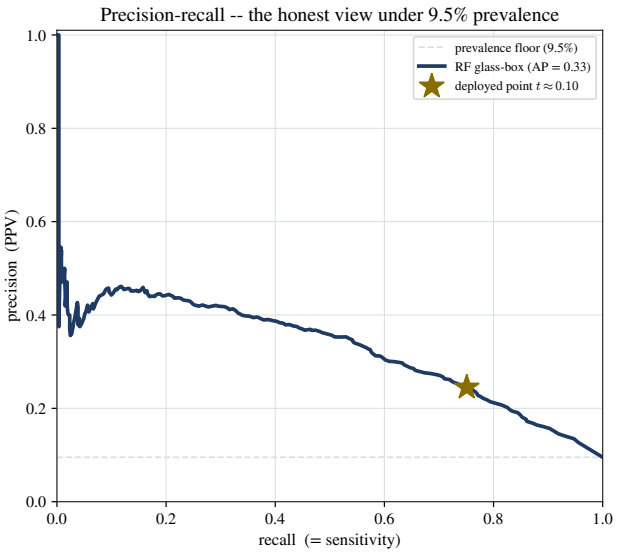

Figure 20. Precision–recall, the honest view under 9.5% prevalence. AP = 0.33 against a 0.095 floor; the deployed point trades precision for the recall a safety cue needs.

6.2 Do the probabilities mean what they say?

A predicted probability of 0.3 ought to mean a freeze about 30% of the time. The reliability diagram (Figure 21) checks this: points on the diagonal are perfectly calibrated. The forest tracks the diagonal where it has data (the histogram below shows most windows sit at low probability); the sparse high-probability bins are noisy, so they are drawn as unconnected dots rather than joined. Figure 22 shows the same scores as two distributions: calm windows (navy) pile up near zero, freeze windows (red) are pushed right. The threshold is just a vertical cut — and because the two piles overlap, no cut separates them perfectly, which is the real origin of the sensitivity/specificity trade-off.

6.3 Does it generalise to a new wearer?

Pooled numbers can hide a model that works for some patients and fails others. Figure 23 breaks the score down by held-out patient. Most subjects sit near the pooled lines (sensitivity 0.71, specificity 0.84), but the spread is real — a reminder that a freeze detector must be validated *per patient*, not just pooled. Two subjects have no labelled freeze windows, so only their specificity is defined.

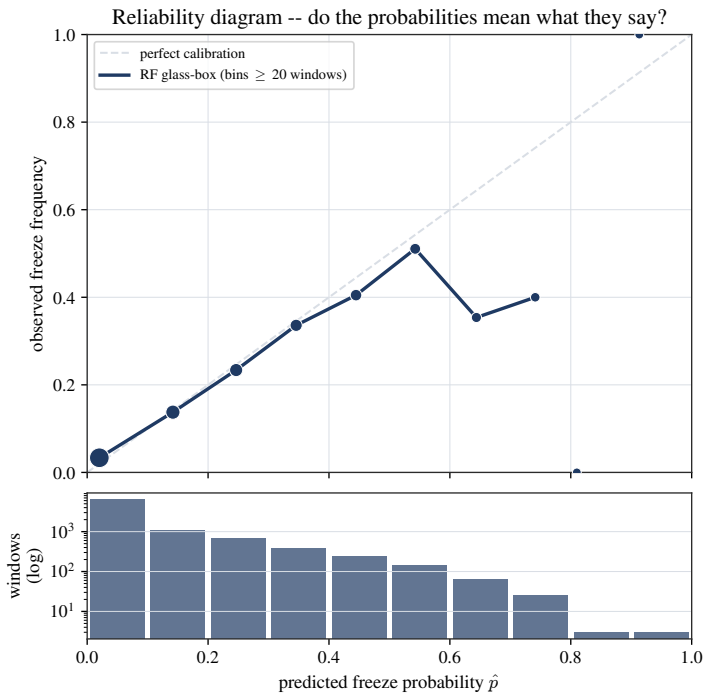

Figure 21. Reliability diagram (out-of-fold). Marker size is the number of windows in each probability bin; the line is drawn only through bins with ≥ 20 windows. The detector is well calibrated where the data live (low probabilities); the histogram below confirms how few windows reach high probability.

6.4 What the features look like — and why the Freeze Index is redundant

Finally, two views of the inputs themselves. The correlation heatmap (Figure 24) shows heavy redundancy: the four band-power features are almost perfectly correlated, as are the three magnitude/jerk features. This is precisely why the model can *reconstruct* the hand-built Freeze Index from its ingredients and finds the fixed ratio redundant (the punchline of §4). The box plots (Figure 25) show *why* the detector works at all: on a freeze window the jerk is far higher and tighter, total power is higher, and the dominant frequency shifts — clean, physical separation between calm gait and freeze.

What we deploy, and what we headline. The *deployed* detector is the 1-D CNN; the random forest is the glass box behind these interpretation plots. We headline **sensitivity** 0.71 **/ specificity** 0.84 (CNN, leave-one-subject-out), with the decision threshold chosen by *nested* LOSO so the figures hold for a new patient. The model's strongest learned cue is ankle *jerk* — and it has, sensibly, learned to outgrow the textbook Freeze Index.

Reproducibility. Every number and figure is generated from the public Daphnet FoG benchmark (ankle sensor) by:

- `colab/fog_allinone.py` — CNN training & leave-one-subject-out predictions;
- `pi/make_accuracy_figs.py` — confusion matrix, metric panel, accuracy caveat, SHAP, permutation importance, partial dependence;
- `pi/make_cnn_figs.py` — the real FoGNet training curve plus the loss/sigmoid/gradient-descent schematics and the ROC, precision–recall, calibration, score-separation, per-subject and feature diagnostics;
- `pi/make_decision_figs.py` — decision contour, parameter grid, operating point;
- `colab/pick_threshold.py` — nested-LOSO threshold selection.

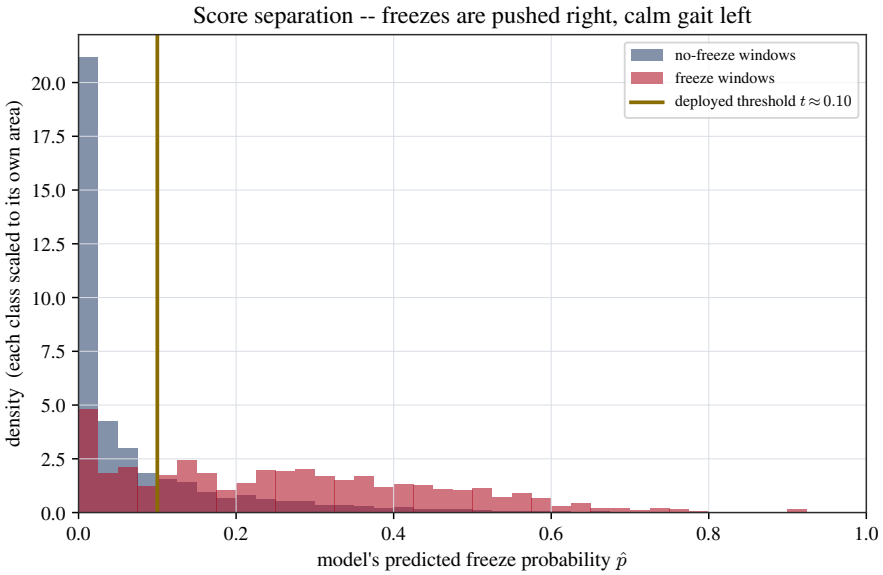

Figure 22. Score separation. The two classes' predicted-probability distributions (each scaled to its own area). Freeze windows (red) are pushed right, calm windows (navy) cluster near zero; the deployed threshold ($t \approx 0.10$, gold) is the cut. The overlap is exactly why a single threshold cannot be both perfectly sensitive and perfectly specific.

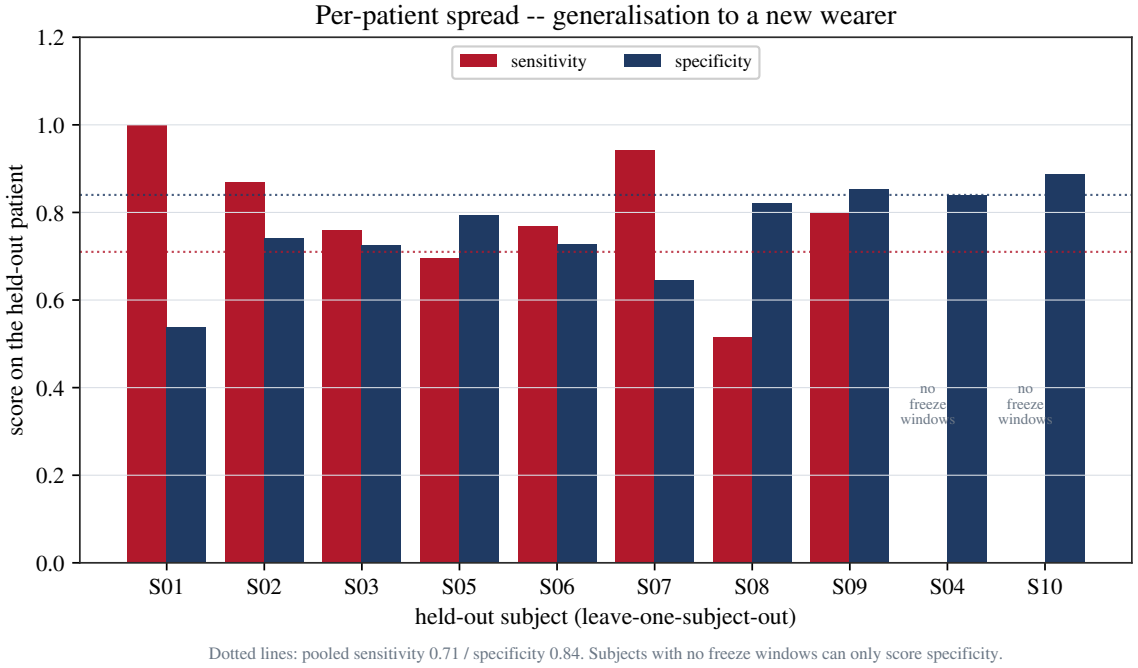

Figure 23. Per-patient sensitivity and specificity under leave-one-subject-out. Dotted lines are the pooled headline (0.71 / 0.84). The spread across patients is the honest picture of how the garment will behave on an unseen wearer.

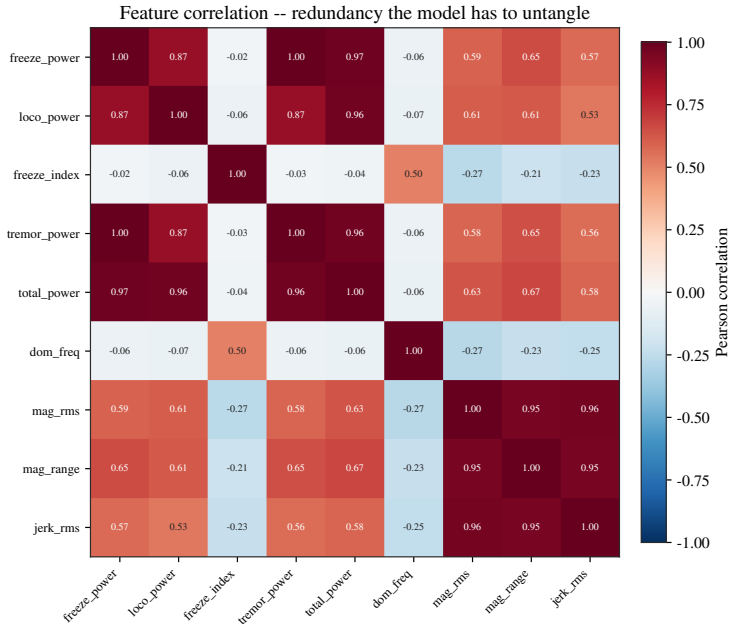

Figure 24. Pearson correlation between the nine interpretable features. Dark-red blocks are near-duplicate features (the band powers; the magnitude/jerk group); the Freeze Index correlates only weakly with anything, because its information is already carried by its separate ingredients.

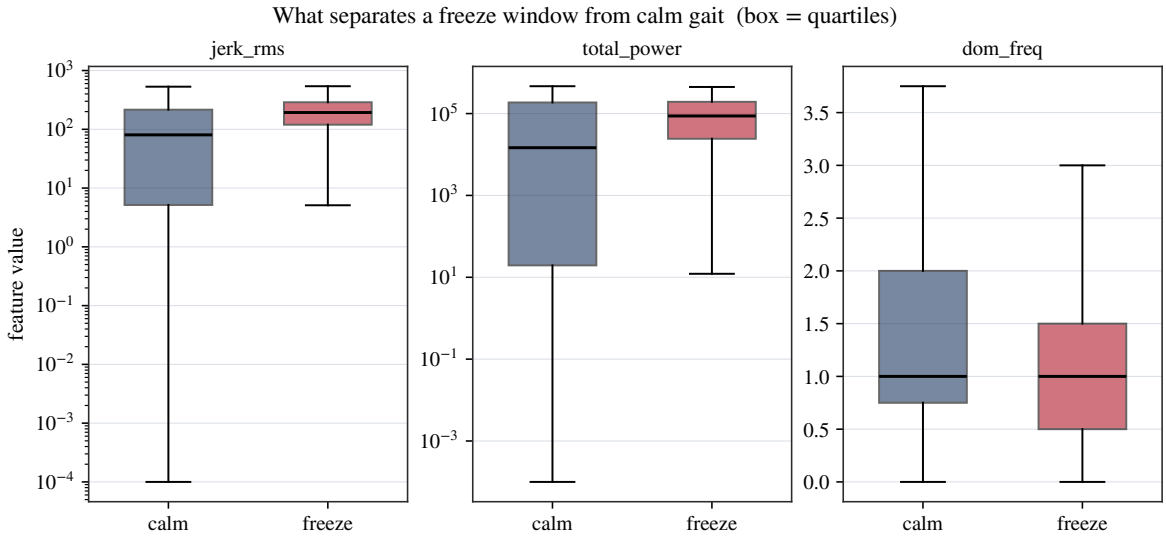

Figure 25. Class-conditional box plots (quartiles, whiskers to the 5th/95th percentile) for three telling features. Jerk and total power are log scaled. The freeze boxes (red) are clearly shifted from the calm boxes (navy) — this separation is what every model in this document is exploiting.